EMPIRICAL TRADE AND INTERNATIONAL FINANCE

Course Number: 020106D05

Meeting Time: Monday 14:00-16:40, Week 6-16

Room: Anzhong Building 201

Instructor: Yuchen Shao

Email: shaoyc@nju.edu.cn

Course Description

- This applied-oriented course studies cutting-edge researches on trade and interactional finance
  - Comparative advantage
  - Gravity equation
  - Extensive/intensive margins of trade
  - Global value chain
  - Chinese import competition
  - TFP estimation
  - Innovation
  - Chinese firm-level data investigation

Course Description (Con't)

- Introduce the latest econometric methodologies and how to apply them in *STATA*
  - Panel data with fixed effects
  - Binary and truncated models and marginal effects
  - Instrumental variables: reverse causality
  - Heckman selection model: selection bias
  - Differences-in-differences
  - Regression discontinuity
  - Propensity scoring matching
  - Quantile regression
- The best way to learn is REPLICATION!

Pre-requisite

- Some knowledge of trade and econometrics
- Professor Ye Zhang's course

- References (optional)
  - Wooldridge, J., *Introductory Econometrics: A Modern Approach*
  - *Handbook of International Economics*
  - Cameron, A. and P. Trivedi, *Microeconometrics Using Stata*
  - Angrist, J. and J. Pischke, *Mostly Harmless Econometrics*
    http://www.mostlyharmlesseconometrics.com/book-contents/

Grading

- Class participation: 10%
  - Read paper before lectures
  - Attendance
- Paper presentation: 90%
  - I assign the paper
  - Group of 2-3 people
  - 15-20 slides
- No exam!

Tentative Schedule

- Week 6 (April 9): Data Processing Basics
- Week 7 (April 16): Regression Basics
- Week 8 (April 23): Paper 1
- (April 28): Class cancelled, seminar time to be announced
- Week 9 (April 30): National holiday, no class
- Week 10 (May 7): Paper 2
- Week 11 (May 14): Paper 3
- Week 12 (May 21): Paper 4
- Week 13 (May 28): Paper 5
- Week 14 (June 4): Paper 6
- Week 15 (June 11): Student Presentations
- Week 16 (June 16): National holiday, no class

Where to get data?

- **Aggregate cross-country data**
  - CEPII: trade, distances, etc http://www.cepii.fr/anglaisgraph/bdd/bdd.htm
  - + BACI dataset if access to COMTRADE http://www.cepii.fr/anglaisgraph/bdd/baci.htm
  - See Santos Silva and Tenreyro (2006) website on gravity equations: good exercises http://privatewww.essex.ac.uk/~jmcss/LGW.html
  - World Bank Doing Business http://www.doingbusiness.org
  - Heritage foundation http://www.heritage.org/Index/Excel/DownloadRawData.xls
  - Haveman's website for sector classification and Rauch's index http://www.macalester.edu/research/economics/PAGE/HAVEMAN/Trade.Resources/TradeData.html

- Peter Schott's website: http://faculty.som.yale.edu/peterschott/sub_international.htm
- NBER website: http://www.nber.org/data/
- World Development Indicators: http://data.worldbank.org/data-catalog/world-development-indicators
- UNCTAD: http://unctad.org/en/Pages/Home.aspx
- Nathan Nunn's webpage, contains country and sector variables (see QJE 2007 paper): http://www.economics.harvard.edu/faculty/nunn/data_nunn
- Penn Tables http://pwt.econ.upenn.edu/php_site/pwt_index.php
- International Financial Statistics (IMF) http://www.imfstatistics.org/imf/
- UNIDO: http://www.unido.org/
- COMTRADE: http://comtrade.un.org/

Where to get data?

- **Country specific data**
  - US: BEA website: provides aggregate trade data on multinational firms http://www.bea.gov/international/index.htm
  - Brazil: Muendler's webpage on Brazil: prices, sector classification, etc. http://www.econ.ucsd.edu/muendler/html/brazil.html
  - IBGE webpage for municipality-level data (useful with Census data) http://www.ibge.gov.br/servidor_arquivos_est/
  - PNAD and Census micro-data are public (but no internet access)
  - IPEA data: www.ipeadata.gov.br/
  - Latin America: Rodrigo Paillacar's website (provides links to access public data) http://team.univ-paris1.fr/teamperso/paillacar/data.htm
  - Enterprise surveys: firm-level data on 100 countries – but messy (except for Thailand?) http://www.enterprisesurveys.org/

Where to get data?

- **China datasets**
  - China data online: http://chinadataonline.org/
  - China Manufacturing Enterprise Survey
  - China Customs Dataset
  - CHIP, CHNS, etc.
  - Sandra Poncet's website (data on distances, etc.)
    http://team.univ-paris1.fr/teamperso/sponcet/index.htm#Donnees

- http://unstats.un.org/unsd/cr/registry/regdnld.asp?Lg=1

- http://www.princeton.edu/~reddings/TradePhd.htm

- WIOD: http://www.wiod.org/new_site/home.htm

STATA BASICS

STATA Windows

- Install STATA
- Open STATA, you will get four windows:
  - Results
  - Review
  - Variables
  - Command
- Use a workspace!
  - submit “cd” (change directory) to set workspace
  - use "/Users/xxx.dta”, clear
- Do file

Useful Codes

- brow
- list in 1/4
- des
- tab var
- sum
- rename
- label variable name “xxx”
- label define inputlabel 0 "Not input" 1 "Input"
- label values intermediate inputlabel

- gen
  - gen lnvar = ln( )
- replace
- recode
- codebook
- drop
- keep
- save xxx.dta, replace
- q
- help
- search

The if Modifier and Logical Expressions

- The if modifier can be used on almost all STATA commands
- Requires a "logical expression" or true/false statement - must include both a variable name and an expression
- Learn these:
  - == Equal to
  - != Not equal to
  - <= Less than or equal to
  - >= Greater than or equal to
  - < Less than
  - > Greater than
  - & AND
  - | OR
- Can use the logical statement to create a dummy variable - NO if clause

Creating Dummy Variables

- Option 1

```
gen approved = 0
replace approved = 1 if action == 1
replace approved = 1 if action == 2
```

- Option 2

```
gen approved = 0
replace approved = 1 if action == 1 | action == 2
```

- Option 3

```
gen approved = 0
replace approved = 1 if action < 3
```

- Option 4

```
gen approved = 0
replace approved = 1 if action <= 2
```

- Option 5

```
gen approved = action < 2
```

- Option 6

```
recode action, (3=0) (2=1) (1=1), gen(approved)
```

Missing Data

- Missing data must be properly coded as "."
- More than one reason for missing, can use .a, .b, etc.
- NOTE: "." is infinitely large when STATA answers logical expressions

Descriptive Stats

- sum varlist, detail
- sutex logkc2 logrder extfin, file(sum_kctc.tex) labels minmax nobs replace
- tabstat varlist, statistics() columns()
- corr
- Several options
  - If modifier
  - “by:” prefix
  - “, by()” option if available

Cross-Tabulations

- Useful for categorical data
- More efficient than "bysort cat1: tab cat2"
- Important options
  - ", col" ",row" - Display percentage within row/column
  - ", nofreq" - Omit raw frequencies
  - ", missing" - Include "missing" as a category in the table

Professional Graphs

- histogram [variable], bin(#) width(#) by(groupvarname)
- kdensity
- “, title(“”)” -- adds a title
- “, note(“”)” -- adds a note
- “, scheme()” – change the color scheme
- graph twoway (scatter rd_ratio tfpacf), (lfit rd_ratio tfpacf), ytitle("R&D intensity") legend(order(1 "Linear fitted values" 2 "Quadratic fitted values"))

- reg wage exper exper2, robust
- predict wagehat
- scatter wage exper || line wagehat exper, sort

- regress pc rd dummyyear*, robust
- acprplot rd, lowess
- graph export pctech.png, replace
- graph combine graph1.gph graph2.gph, col(#) row(#)

Basic Macros

- global macroname var1 var2
- $macroname
- Example:
  - global inter1 tfpacf_lagipr1 fie_lagipr1 ord_lagipr1 export_lagipr1 age_lagipr1 lemploy_lagipr1
  - quietly reg rd_ratio tfpacf ord_ratio lagipr1 fie export_ratio age lemploy $inter1 dummy*, vce(cluster id)
  - outreg2 using olsrd1.tex, tex label se drop(dummy*) addtext(Year FE, Yes, Industry FE, Yes, Province FE, Yes) coefastr dec(4) append

Looping Preliminaries

- Loops repeat a series of commands with minor variations, without excessive cutting and pasting.
- Types of Loops
  - foreach
  - forvalues
  - while

foreach

- Allows you to loop over multiple things
  - varlist
  - numlist
  - macro
  - more
- Syntax:

```
foreach lname [of/in] [type of list]{
command 1
command 2
etc.
}
```

forvalues

- Loops over range of numbers
- Syntax:

```
forvalues  lname = range{
command 1
command 2
etc.
}
```

- Less flexible than foreach, but potentially useful

Forvalues example

- * GDP annually 1960-2016 GDP at market prices (constant 2010 US$) world bank
- insheet using "/Users/Yuchen/Documents/Research/data/World Bank/gdp2010.csv", clear
- rename v4 iso3
- ren v3 country
- drop v1 v2 v62
- forvalues i=5/61{
- ren v`i' gdp`i’
- }

ADVANCED DATA MANAGEMENT

Excel to STATA

1. Use Excel to "Save As" - comma separated values (.csv) in workspace
2. Get STATA to read it in using "insheet". Syntax:

   ```
   insheet using filename.csv
   ```

3. First row in spreadsheet becomes variable names

insheet example

- clear all
- cd "/Users/Yuchen/Documents/Research/5KnowledgeCapital/KCnaics”
- set more off
- insheet using "/Users/Yuchen/Documents/Research/5KnowledgeCapital/KCdata/sic7287.txt", delimit(" ") clear
- keep v1 v2
- rename v1 sic72
- rename v2 sic87
- codebook
- tostring sic72, replace
- tostring sic87, replace
- gen length_sic72 = length(sic72)
- tab length_sic72
- drop length*
- sort sic87
- save sic72_sic87.dta, replace

STATA to Excel

1. Use the "outsheet" command. Syntax:

   ```
   outsheet using filename.csv
   ```

2. Use Excel's Import wizard. Data → Text to Columns, if necessary.

Plain Text to STATA

1. Data probably look like:

```
2007010000000400881008703311001007104250 0
2007010000000400881006602521001007300000 0
2007010000000400881006900916001002099999 9
2007010000000400881007000726047002099999 9
2007010000000400881007200226001000099999 9
2007010000000500281002708425028003300000 0
```

2. Need a dictionary and an "infix" command

```
infix 1-4 year 5-6 datanum ... using filename.raw
```

egen is awesome

- "egen" is full of amazing data capabilities

  `egen newvar =`

- We'll cover my favorites - not complete treatment. Before you decide it is impossible to do something that you want to do, try "help egen"
- Reminder: "by" prefix: Many egen commands are most useful in combination with the "by varlist :" prefix.

Examples:

```
egen id = group(iso3)
egen id = group(state industry)
egen va_mean = mean(valueadd), by(naics)
by id2: egen inputnum = count(hs6)
```

Use egen commands

- stats commands (mean, max, min, median, sd, rank, etc.) - Especially useful with a by prefix and a group variable. Lets you add a variable with some group statistics to the individual record. Example:

  ```
  bysort villageid: egen villmedinc = median(income)
  ```

- row commands (rowmean, rowsd, rowmax, etc.) - Looks across variables to calculate statistics within an observation. Example:

  ```
  egen highscore = rowmax(readscore writescore mathscore)
  ```

- fill - creates a pattern of repeating values in a dataset.

Merging Basics

- Types of Merges
  - One-to-one
  - One-to-Many
- Always need linking variable(s) - same variable names, coded identically in both datasets.
- Good habit to have both datasets saved sorted by linking variables. No longer mandatory in most recent version of STATA.

Merge Command

- Syntax:

  ```
  merge linkvar using dataset
  ```

- Terminology - STATA calls the two datasets "Master" and "Using"
- Always check your "_merge" variable
  - _merge = 3 - successful match
  - _merge = 2 - this value of "linkvar" was not found in "Master"
  - _merge = 1 - this value of "linkvar" was not found in "Using"
- Additional options: ",keep" and ", _merge()" can be useful

Collapse

- Collapse calculates "statistics" for subgroups and creates a new dataset with subgroups as observations and "statistics" as variables.
- "statistics" can include mean, sum, etc.
- Syntax:

```
collapse (stat) varname (stat) varname.. [pw=weight], b
```

Collapse Example

- Data at the (state $\times$ industry $\times$ year)-level with employment totals at the four digit industry level. You want it at the two-digit level.
- ```
  gen ind2dig = floor(ind4dig/100)
  collapse (sum) employment, by (state year ind2dig)
  ```
- National Annual 2-digit employment totals

  ```
  collapse (sum) employment, by (ind2dig year)
  ```
  ```

Example

- * imported input data aggregate into country-year level
- use "/Users/Yuchen/Documents/Research/data/COMTRADE/Zhu/Baci96_2002.dta", clear
- forvalues i=3/6 {
- append using "/Users/Yuchen/Documents/Research/data/COMTRADE/Zhu/Baci96_200`i'.dta"
- }
- sort hs6 t
- codebook hs6 //5111
- tostring hs6, gen(hs96) format(%06.0f)

- * aggregate up to j(importer) t hs96 level
- egen id = group(t hs96 j)
- collapse (firstnm) t hs96 j (sum) v, by(id)
- drop id
- save Baci0206_importer.dta, replace

- *aggregate up to i(exporter) j(importer) t level
- egen id = group(t i j)
- collapse (firstnm) t i j (sum) v, by(id)
- drop id
- save Baci0206_bilateral.dta, replace

- sort j
- merge m:m j using "/Users/Yuchen/Documents/Research/data/COMTRADE/Zhu/countrycode2.dta”
- drop if _merge == 1
- drop name_english _merge
- sort I
- merge m:m i using "/Users/Yuchen/Documents/Research/data/COMTRADE/Zhu/countrycode1.dta”
- drop if _merge == 1
- drop name_english _merge

- egen v_mean = mean(v), by(i)
- save Baci0008_bilateral.dta, replace

Long or Wide?

- There are two ways to store panel data.
- Long looks like this

```
id year price
1  2005 70
1  2006 75
1  2007 79
2  2005 80
2  2006 85
2  2007 89
etc.
```

- Multiple observations per id. One variable per x.

Long or Wide? (Con't)

- Wide looks like this

```
id price2005 price2006 price2007
1  70        75        79
2  80        85        89
etc.
```

- Single observation per id. Multiple variables per x
- STATA panel commands want your data long. Consistent with how we write the equations for a panel model:

$$y_{ijt} = X_{ijt}\beta + \delta_t + \gamma_j + \epsilon_{ijt}$$

Reshape

- "reshape" switches data from long to wide or vice versa
- Syntax:

```
reshape long stubname1 stubname2, i(idvar) j(timevar)
```

- If no id variable exists use egen. Example: you need ids for each state $\times$ industry, use "egen id $=$ group(state industry)"

reshape long example

- insheet using "/Users/Yuchen/Documents/Research/12KnowledgeCapital/KCdata/gdp.csv", clear
- keep v2 v41-v53
- rename v2 iso3
- drop if iso3 == "Country Code”
- rename v41 gdp1996
- rename v42 gdp1997
- rename v43 gdp1998
- rename v44 gdp1999
- rename v45 gdp2000
- rename v46 gdp2001
- rename v47 gdp2002
- rename v48 gdp2003
- rename v49 gdp2004
- rename v50 gdp2005
- rename v51 gdp2006
- reshape long gdp, i(iso3) j(year)
- label variable gdp "GDP”
- drop if gdp == .
- sort iso3 year
- save gdp_9606.dta, replace

reshape wide example

- use "/Users/Yuchen/Documents/Research/data/Patent/Citation/patsic06_mar09_ipc.dta", clear
- contract patent appyear gyear icl_class
- sort patent //duplicates insides which belongs to different IPC classes
- quietly tab appyear //1967 to 2006
- drop _freq
- bysort patent: gen num = _n
- reshape wide icl_class, i(patent) j(num)

One difficulty I recently met

- insheet using "/Users/Yuchen/Documents/Research/data/migration/migration_origin.csv", delimiter(",") clear
- forvalues j=8/241{
- quietly replace v`j' = subinstr(v`j',",","",.)
- local try = strtoname(v`j'[1])
- capture rename v`j' country`try'
- }
- drop if missing(typeofdataa)
- drop sortorder notes typeofdataa
- ren majorarearegioncountryorareaofde country_nber
- drop countryorareaoforigin countryOther_North countryOther_South
- drop v*
- quietly destring, force replace
- drop country_nber code
- ren Year year
- sort year collapse (sum) country* , by(year)
- reshape long country, i(year) j(n) string

xtset

- Once data are long, and you have an id variable, tell STATA about it
- Syntax:

```
xtset idvar timevar, delta()
```

- Make sure your time info is stored in a single variable in STATA format. See "help dates" Example:

```
gen date = ym(year, month)
```

Lags and Differences

- Once STATA knows you have panel data, you can use built-in features to create new variables based on leads, lags, and changes.
  - l#.varname = lag of # periods
  - f#.varname = lead of # periods
  - d#.varname = current value minus lag of # periods
- Examples:

```
gen pricelag = l.price
gen annualchgprice = d12.price
```

Example:

- *Use perpetual method to calculate capital stock, like Brandt et al. (2012) depreciation rate 9%
- Similar to Maskus and Yang (Canadian Journal of Economics, forthcoming)

We define the ratio of capital stock in industry j to value added as the measure of capital intensity. We estimate capital stock figures in each U.S. industry from 1995 to 2010 by the perpetual inventory method using capital expenditure data from the U.S. Census of Manufactures. The annual physical capital stock of industry j is determined as follows:

$$K_{j,t} = I_{j,t-1} + (1-\delta)K_{j,t-1}$$

where δ is the depreciation rate, assumed to be 6%.[8] The variables $I_{j,t-1}$ and $K_{j,t-1}$ are, respectively, capital expenditures and capital stocks in the previous year. The initial capital stock of each industry is estimated by

$$K_{j0} = \frac{I_{j0}}{(g_j+\delta)}$$

where g_j is the average annual growth rate of capital expenditure in sector j. The initial year is 1970.

- //deflator base year: 1987; capital stock base year: 1987
- use "/Users/Yuchen/Documents/Research/data/USCensusManufacturers/naics5811.dta", clear
- keep naics year invest vadd piship piinv
- drop if invest == .
- gen real_invest = invest/piinv
- gen real_vadd = vadd/piship
- xtset naics year
- xtdes
- xtbalance, range(1958,2006)
- xtdessum
- save naics_invest_5808.dta, replace

- ////gen annual growth rate from 1992 to 2000: a lot of negative growth rates
- use naics_invest_5808.dta, clear
- keep naics year real_invest
- keep if year == 1958 | year == 2000
- reshape wide real_invest, i(naics) j(year)
- gen capp_growth = (real_invest2000/real_invest1958)^(1/(2000-1958))-1
- sum if capp_growth < 0 //82
- sum //452
- keep naics capp_growthsave capp_growth.dta, replace
- //generate initial capital stock in 2000
- use naics_invest_5808.dta, clear
- keep if year == 2000
- keep naics year real_investsort naics
- merge m:m naics using capp_growth.dta
- gen capp_initial = real_invest/(capp_growth+0.09)
- keep naics year capp_initial
- save capp_initial.dta, replace

- //calculate capital stock every year
- use naics_invest_5808.dta, clear
- xtset naics year
- xtdesxtbalance, range(2000,2006)
- drop vadd invest piship piinv
- sort naics year
- merge m:m naics year using capp_initial.dta
- sort naics year
- xtset naics year
- replace capp_initial = real_invest + 0.91 * l.capp_initial if year == 2001
- replace capp_initial = real_invest + 0.91 * l.capp_initial if year == 2002
- replace capp_initial = real_invest + 0.91 * l.capp_initial if year == 2003
- replace capp_initial = real_invest + 0.91 * l.capp_initial if year == 2004
- replace capp_initial = real_invest + 0.91 * l.capp_initial if year == 2005
- replace capp_initial = real_invest + 0.91 * l.capp_initial if year == 2006
- replace capp_initial = real_invest + 0.91 * l.capp_initial if year == 2007
- replace capp_initial = real_invest + 0.91 * l.capp_initial if year == 2008
- drop _merge

Translate

- * deal with hitech industries
- insheet using hitech.csv, clear
- save hitech.dta, replace
- //clear all
- //unicode analyze hitech.dta
- //quietly unicode encoding set GB2312 /*GB2312应该就是Windows系统的编码方式*
- ///quietly unicode translate hitech.dta, transutf8 invalid
- //unicode retranslate hitech.dta, transutf8 invalid
- use hitech.dta, clear
- tostring v2, gen(industry)
- gen lengthind = length(industry)
- keep if lengthind == 4
- contract industry
- drop _freq
- save hitechind.dta, replace

Check Duplicates

- duplicates report panelid
- duplicates drop //in terms of all variables
- sort panelid
- quietly by panelid: gen dup = cond(_N==1,0,_n)
- tab dup
- drop if dup > 0 //(224 observations deleted)
- drop dup
- duplicates report panelid

Additional Codes

- contract hs_id party_id company hs year
- quietly unique city, by(company) gen(city_num)
- bysort party_id: gen company_num = _n
- bysort company: keep if _n = _N //保留最后一个obs
- gen hs6 = floor(hs8/100)

Deal with String Variables

- tostring, force
- destring
- subinstr("this is the day","is","X",1) = "thX is the day"
- subinstr("this is the hour","is","X",2) = "thX X the hour"
- subinstr("this is this","is","X",.) = "thX X thX"
- replace company = subinstr(company," ","",.)
- replace company = subinstr(company,"市","",.)
- drop if regexm(company, "出口")
- gen city = substr(party_id, 1, 4)
- replace party_id=upper(party_id)
- gen byte notnumeric = real(hs_id)==.
- tab notnumeric

- Example 1: Extracting zip codes from addresses
- input str60 address
- "4905 Lakeway Drive, College Station, Texas 77845 USA"
- "673 Jasmine Street, Los Angeles, CA 90024"
- "2376 First street, San Diego, CA 90126"
- "6 West Central St, Tempe AZ 80068"
- "1234 Main St. Cambridge, MA 01238-1234"
- end
- gen zip = regexs(0) if(regexm(address, "[0-9][0-9][0-9][0-9][0-9]"))
- list

```
     +-----------------------------------------------------------+
     |                                              address   zip |
     |-----------------------------------------------------------|
  1. | 4905 Lakeway Drive, College Station, Texas 77845 USA 77845 |
  2. |            673 Jasmine Street, Los Angeles, CA 90024 90024 |
  3. |               2376 First street, San Diego, CA 90126 90126 |
  4. |                    6 West Central St, Tempe AZ 80068 80068 |
  5. |               1234 Main St. Cambridge, MA 01238-1234 01238 |
     +-----------------------------------------------------------+
```

- Example 1, Variation Number 1
- clear
- input str60 address
- "4905 Lakeway Drive, College Station, Texas 77845"
- "673 Jasmine Street, Los Angeles, CA 90024"
- "2376 First street, San Diego, CA 90126"
- "66666 West Central St, Tempe AZ 80068"
- "12345 Main St. Cambridge, MA 01238"
- end
- gen zip = regexs(0) if(regexm(address, "[0-9][0-9][0-9][0-9][0-9]"))
- list

- gen zip = regexs(0) if(regexm(address, "[0-9][0-9][0-9][0-9][0-9]$")) list

- Example 1, Variation Number 2
- clear
- input str60 address
- "4905 Lakeway Drive, College Station, Texas 77845 USA"
- "673 Jasmine Street, Los Angeles, CA 90024"
- "2376 First street, San Diego, CA 90126"
- "66666 West Central St, Tempe AZ 80068"
- "12345 Main St. Cambridge, MA 01238-1234"
- "12345 Main St Sommerville MA 01239-2345"
- "12345 Main St Watertwon MA 01239 USA"
- end
- gen zip = regexs(1) if regexm(address, "([0-9][0-9][0-9][0-9][0-9])[\-]*[0-9]*[a-zA-Z]*$")
- list

- [\-]* - matching zero or more dashes "-"
- [0-9]* - matching zero or more numbers
- [a-zA-Z]* - matching zero or more blank spaces or letters

- Example 2: Extracting first name and last name and switching their order
- clear
- input str40 fullname
- "John Adams"
- "Adam Smiths"
- "Mary Smiths"
- "Charlie Wade"
- end
- gen n = regexs(2)+", "+regexs(1) if regexm(fullname, "([a-zA-Z]+)[ ]*([a-zA-Z]+)")
- list
- **([a-zA-Z]+) -** subexpression capturing a string consisting of letters, both lower case and upper case. This will be the first name.
- **[ ]* -** matching with space(s). This is the spacing between first name and last name.
- **([a-zA-Z]+)** - subexpression capturing a string consisting of letters. This will be the last name.

- Example 3: Two- and four- digit values for year.
- input str18 date
- 20jan2007
- 16June06
- 06sept1985
- 21june04
- 4july90
- 9jan1999
- 6aug99
- 19august2003
- end
- gen day = regexs(0) if regexm(date, "^[0-9]+")
- gen month = regexs(0) if regexm(date, "[a-zA-Z]+")
- gen year = regexs(0) if regexm(date, "[0-9]*$")
- replace year = "20"+regexs(0) if regexm(year, "^[0][0-9]$")
- replace year = "19"+regexs(0) if regexm(year, "^[1-9][0-9]$")
- gen date2 = day+month+year

[ ]	Square brackets indicate that one of the characters inside the brackets should be matched. For example, if I wanted to search for a single letter between f and m, I would type "[f-m]"
a-z	A range specifies that any value within that range is acceptable. This is case sensitive, so a-z is not the same as A-Z, if either case can be counted as a match, include both a-zA-Z. Numeric values are also acceptable as ranges (e.g. 0-9).
.	A period matches any character.
\	Allows you to match characters that are usually regular expression operators. For example, if you wanted to match a "[" you would type \[instead of just a single [.
*	Match zero or more of the characters in preceding expression. For example if I wanted to match a number made up of one or more digits if there is a number, but still want to indicate a match if the rest of the expression fits, I could specify [0-9]*
+	Match one or more of the characters in the preceding expression. For example if I wanted to match a word containing any combination of letters, I would specify [a-zA-Z]+
?	Match either zero or one of the previous expression.
^	When it appears at the beginning of an expression, a "^" indicates that the following expression should appear at the beginning of the string.
$	When it appears at the end of an expression, a "$" indicates that the preceding expression should appear at the end of the string. For example, if I wanted to match a number that was the last thing to appear at the end of a string, I would specify "[0-9]+$"
|	The logical operator or, indicating that either the expression preceding it or following it qualify as a match.
( )	Creates a subexpression within a larger expression. Useful with the "or" perator (i.e. |), and when extracting and replacing values. For example, if I wanted to extract a numeric value which I know follows directly after a word or set of letters, I could use the regular expression "[a-zA-Z]+([0-9]+)" this matches the whole expression, but allows you to select the portion in the parentheses (called a substring). Handling substrings is discussed in greater detail below.

- display regexm("907-789-3939", "([0-9]*)\-([0-9]*)\-([0-9]*)")
- display regexs(0)
- display regexs(1)
- display regexs(2)
- display regexs(3)

- **regexm** - used to find matching strings, evaluates to one if there is a match, and zero otherwise
- **regexs** - used to return the *n*th substring within an expression matched by regexm (hence, regexm must always be run before regexs, note that an "if" is evaluated first even though it appears later on the line of syntax).
- **regexr** - used to replace a matched expression with something else.